Write a Program to implement web Service to Convert temperature from Fahrenheit to Celsius. Use following formula for conversion. Formula: C = (°F - 32) × 5/9

STEP 1: OPEN NETBEANS

- Start **Apache NetBeans**

STEP 2: CREATE NEW PROJECT

- Click **File**
- Click **New Project**

STEP 3: SELECT PROJECT TYPE

- Select **Java Web**
- Select **Web Application**
- Click **Next**

STEP 4: PROJECT DETAILS

- Project Name:
  TemperatureWebService
-
- Click **Next**

STEP 5: SERVER & VERSION

- Server: **GlassFish Server**
- Java EE Version: **Java EE 7**
- Click **Next**
- Click **Finish**

STEP 6: CREATE WEB SERVICE

- Right-click **Source Packages**
- Click **New**
- Click **Web Service**

STEP 7: WEB SERVICE DETAILS

- Web Service Name:
  TemperatureService
-
- Package:
  com.temp
-
- Click **Finish**

STEP 8: WRITE CODE

Delete old code and paste this:

```
package com.temp;

import javax.jws.WebService;
import javax.jws.WebMethod;

@WebService
public class TemperatureService {

    @WebMethod
    public double fahrenheitToCelsius(double f) {
        return (f - 32) * 5 / 9;
    }
}
```

STEP 9: SAVE FILE

- Press **Ctrl + S**

STEP 10: CLEAN & BUILD

- Right-click project
- Click **Clean and Build**

STEP 11: RUN PROJECT

- Right-click project
- Click **Run**

STEP 12: TEST WEB SERVICE

- Right-click `TemperatureService`
- Click **Test Web Service**
- Enter:
  98.6
-
- Click **fahrenheitToCelsius**

OUTPUT

- 37

Program to Implement Web Service to Calculate Factorial of a Given Number

STEP 1: Open NetBeans

- Start **Apache NetBeans**

STEP 2: Create New Project

- Click **File → New Project**

- Select **Java Web**
- Select **Web Application**
- Click **Next**

STEP 3: Project Details

Project Name:
FactorialWebService

-
- Click **Next**
- Server: **GlassFish Server**
- Java EE Version: **Java EE 7**
- Click **Finish**

STEP 4: Create Web Service

- Right-click **Source Packages**
- Click **New → Web Service**

STEP 5: Web Service Details

Web Service Name:
FactorialService

-

Package:
com.factorial

-
- Click **Finish**

STEP 6: WRITE THE PROGRAM CODE

Replace the generated code with the following:

```
package com.factorial;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;
```

```
@WebService
public class FactorialService {

    @WebMethod
    public long factorial(@WebParam(name = "number") int number) {

        long fact = 1;
        for (int i = 1; i <= number; i++) {
            fact = fact * i;
        }
        return fact;
    }
}
```

STEP 7: SAVE THE FILE

- Press **Ctrl + S**

STEP 8: CLEAN & BUILD

- Right-click project
- Click **Clean and Build**

STEP 9: RUN THE PROJECT

- Right-click project
- Click **Run**
- GlassFish server will start

STEP 10: TEST WEB SERVICE

- Right-click **FactorialService**
- Click **Test Web Service**

Enter number:
5

Click **factorial**

Create a Web Application to find Multiplication of Two Numbers using Simple SOAP Web Service

---

1. WHAT YOU ARE GOING TO BUILD

You will create:

1. **SOAP Web Service** → performs multiplication
2. **Web Application (JSP)** → takes two numbers from user
3. JSP calls SOAP service → displays result

---

2. REQUIRED SOFTWARE

1. **Apache NetBeans IDE**
2. **JDK 8** (IMPORTANT – SOAP works smoothly with JDK 8)
3. **GlassFish Server** (comes with NetBeans)

3. CREATE SOAP WEB SERVICE PROJECT

STEP 1

Open **NetBeans**

Click
File → New Project

STEP 2

Select
Java Web → Web Application

Click **Next**

STEP 3

Fill details:

Field	Value

Project Name	`Multiplication SOAP`
Server	GlassFish Server
Java EE Version	Java EE 7 or 8

Click **Finish**

4. CREATE SOAP WEB SERVICE

STEP 4

Right-click on **Source Packages**

Click
New → Web Service

STEP 5

Fill details:

Field	Value
Web Service Name	`MultiplyService`
Package	`service`

Click **Finish**

5. WRITE SOAP SERVICE CODE

NetBeans will create `MultiplyService.java`

Replace code with this:

```
package service;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;
```

```
@WebService(serviceName = "MultiplyService")
public class MultiplyService {

    @WebMethod(operationName = "multiply")
    public int multiply(
            @WebParam(name = "num1") int num1,
            @WebParam(name = "num2") int num2) {

        return num1 * num2;
    }
}
```

What this does

- `@WebService` → creates SOAP service
- `@WebMethod` → exposes method
- `multiply()` → logic for multiplication

6. TEST SOAP SERVICE

STEP 6

Right-click on `MultiplyService.java`

Click **Run File**

Browser opens showing **WSDL page**

Click
multiply → Test

Enter:

num1 = 5
num2 = 6

Click **Test Method**

Output:

30

SOAP service is working ✔

7. CREATE JSP CLIENT PAGE

STEP 7

Right-click on **Web Pages**

Click
New → JSP

Name it:
`index.jsp`

8. WRITE JSP CODE (USER INTERFACE)

```
<%@ page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
   <title>SOAP Multiplication</title>
</head>
<body>

<h2>Multiplication using SOAP Web Service</h2>

<form method="post">
   Enter Number 1:
   <input type="text" name="n1"><br><br>

   Enter Number 2:
   <input type="text" name="n2"><br><br>

   <input type="submit" value="Multiply">
</form>

<%
   if(request.getParameter("n1") != null && request.getParameter("n2") != null) {

      int a = Integer.parseInt(request.getParameter("n1"));
      int b = Integer.parseInt(request.getParameter("n2"));

      service.MultiplyService_Service service =
            new service.MultiplyService_Service();
      service.MultiplyService port =
            service.getMultiplyServicePort();
```

```
        int result = port.multiply(a, b);

        out.println("<h3>Result = " + result + "</h3>");
    }
%>

</body>
</html>
```

9. HOW JSP CALLS SOAP SERVICE

NetBeans automatically generates:

- SOAP client classes
- WSDL binding
- Stub code

This line connects to SOAP service:

service.MultiplyService port = service.getMultiplyServicePort();

10. RUN THE WEB APPLICATION

STEP 8

Right-click on **Project**

Click **Run**

Browser opens:

http://localhost:8080/MultiplicationSOAP/

Enter numbers → Click **Multiply** → Result displayed

11. SAMPLE OUTPUT

Enter Number 1: 10
Enter Number 2: 5

Result = 50

Create a web application to find addition of a three number using Simple SOAP service.

1. AdditionSOAP.java

```
package service;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

/**
 * author ORISANLAB
 */
@WebService(serviceName = "AdditionSOAP")
public class AdditionSOAP {

    /**
     * This is a sample web service operation
     */
    @WebMethod(operationName = "addition")
    public int addition(
        @WebParam(name = "num1") int num1,
        @WebParam(name = "num2") int num2,
        @WebParam(name = "num3") int num3
    ) {
        return num1 + num2 + num3;
    }
}
```

Create a web application to find square of a number using Simple SOAP service.

2.squareservice.java

```
package service;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

/**
 * @author ORISANLAB-138
 */
```

```
@WebService(serviceName = "squareservice")
public class squareservice {

    /**
     * This is a sample web service operation
     */
    @WebMethod(operationName = "square")
    public int square(
        @WebParam(name = "num1") int num1
    ) {
        return num1 * num1;
    }
}
```

Create a web application to find cube of a number using Simple SOAP service.

3.cubeservice.java

```
package service;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

/**
 * @author ORISANLAB-138
 */
@WebService(serviceName = "cubeservice")
public class cubeservice {

    /**
     * This is a sample web service operation
     */
    @WebMethod(operationName = "cube")
    public int cube(
        @WebParam(name = "num1") int num1
    ) {
        return num1 * num1 * num1;
    }
}
```

Develop application to consume Google's search / Google's Map RESTful Web service

CONSUME GOOGLE MAP REST API

STEP 1: WHAT IS AN API KEY?

- Google gives permission using **API KEY**
- Without key → Google blocks request
- Key = password for Google services

STEP 2: CREATE GOOGLE API KEY

2.1 Open Google Cloud

1. Open browser
2. Go to:

https://console.cloud.google.com/

3. Login using **Gmail account**

2.2 Create New Project

1. Click **Select Project**
2. Click **New Project**
3. Project Name: `GoogleMapAPI`
4. Click **Create**

2.3 Enable Google Maps API

1. Click ☰ **Menu**
2. Click **APIs & Services**

3. Click **Library**
4. Search: `Geocoding API`
5. Click **Enable**

✔ API enabled

2.4 Create API Key

1. APIs & Services → **Credentials**
2. Click **Create Credentials**
3. Select **API Key**
4. Copy the key
   Example:

AIzaSyD3wvBlzZH71NrYH3jLsHYi8rLhix95q08

✔ API KEY READY

STEP 3: CREATE JAVA PROJECT (NETBEANS)

1. Open **Apache NetBeans**
2. File → New Project
3. Java → **Java Application**
4. Project Name: `GoogleMapClient`
5. Finish

STEP 4: CREATE JAVA CLASS

1. Right-click **Source Packages**
2. New → Java Class
3. Name: `GoogleMapService`
4. Finish

STEP 5: WRITE

Paste this FULL CODE

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

public class GoogleMapService {

    public static void main(String[] args) {

        try {
            String place = "Mumbai";
            String apiKey = "PASTE_YOUR_API_KEY_HERE";

            String urlStr =
            "https://maps.googleapis.com/maps/api/geocode/json?address="
            + place + "&key=" + apiKey;

            URL url = new URL(urlStr);
            HttpURLConnection con =
            (HttpURLConnection) url.openConnection();

            con.setRequestMethod("GET");

            BufferedReader br =
            new BufferedReader(new InputStreamReader(con.getInputStream()));

            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

STEP 6: ADD INTERNET PERMISSION

✔ Java desktop app needs **NO extra permission**
✔ Just ensure internet is ON

STEP 7: RUN PROGRAM

1. Right-click `GoogleMapService`
2. Click **Run File**

STEP 8: OUTPUT (WHAT YOU SHOULD SEE)

JSON data like:


```
{
 "results": [
  {
   "geometry": {
    "location": {
     "lat": 19.0760,
     "lng": 72.8777
    }
   }
  }
 ]
}
```


✔ Google Map REST API consumed successfully